

『Starsand Island』 NS2版マウス操作の開発

1. プロジェクト基本情報

項目	内容
タイトル	Starsand Island
プロジェクト概要	Nintendo Switch 2版における、Joy-Con 2を使用したマウス操作モードの開発
対象機能	画面上で建物や物体を選択・配置・編集する機能、メニュー操作、カメラ操作
開発環境	Unity
開発言語	C#
主な技術	Unityの入力基盤、Nintendo Switch 2 SDKのマウス操作API
担当領域	入力基盤の設計・実装、入力方式の切り替え、既存の編集機能・画面操作・カメラ操作への統合、デバッグ支援
公開範囲	CEDEC講演ではクライアントの許諾を得てゲーム映像を使用。ソースコードおよび内部資料は非公開
基本機能デモ	Nintendo Switch 2 マウス操作 基本機能デモ
ゲーム内デモ	Nintendo Switch 2 マウス操作 ゲーム内統合デモ

2. 開発背景と目的

本タイトルは複数のプラットフォームで展開されており、プレイヤーが画面上で建物や物体を選び、配置、移動、回転、削除などを行う編集機能があります。この機能では、対象の指定、複数項目の選択、設定値の変更など、PC版でマウスを使用する操作を多く含みます。Nintendo Switch 2版でも、Joy-Con 2によるマウス操作へ対応し、画面上の対象を直接指し示して直感的に編集できるようにする必要がありました。

一方、Joy-Con 2から得られるマウス入力とはPC版のマウス入力と取得方法が異なるため、既存機能へそのまま接続できませんでした。また、各プラットフォーム向けの編集操作、メニュー操作、カメラ操作、画面上の操作案内がすでに実装されていたため、それらを維持したままNintendo Switch 2固有の入力方式を追加する必要がありました。

そこで、Nintendo Switch 2 SDKから取得した座標やボタン入力を、既存機能が扱える操作へ変換する入力基盤を開発しました。さらに、Joy-Con 2の使用状態、ユーザー設定、プレイヤーが編集機能を使用しているかどうかに応じて、ゲームパッド操作とマウス操作を切り替える仕組みを実装しました。

3. 本人の担当範囲

私は、Nintendo Switch 2向けマウス操作について、SDKからの入力取得から既存の編集機能への統合までを担当しました。主な担当内容は以下の通りです。

- Nintendo Switch 2 SDKのマウス操作APIをUnityから利用する入力拡張クラスの設計・実装
- ユーザー設定、Joy-Con 2の使用状態、編集機能の利用状態を組み合わせた入力方式の判定
- ゲームパッド操作とマウス操作を切り替えた際に、利用中の操作を引き継ぎ、二重入力や操作不能を防ぐ制御
- 物体の配置・編集、カメラ、各種メニュー、画面上の操作案内へのマウス操作対応
- デバイス接続・構成変更への対応と、入力状態を確認するデバッグ機能の実装

4. 全体の処理構成

コントローラーのポインター入力／使用姿勢

↓

Nintendo Switch 2 SDKのマウス操作API

- ・ポインター座標
- ・ボタン状態
- ・水平面上で使用されているかを示す属性

↓

Nintendo Switch 2入力拡張

- ・入力状態の毎フレーム更新
- ・仮想カーソルの制御
- ・デバイス変更の監視

↓

入力モード管理

- ・ユーザー設定との組み合わせ
- ・編集機能を使用しているかどうかの判定
- ・切り替え前後の操作状態の引き継ぎ

↓

既存ゲームシステム

- ・物体の配置／編集
- ・カメラ操作
- ・メニューのクリック／ドラッグ／スクロール
- ・画面上の操作案内

Nintendo Switch 2固有の入力取得と、ゲーム内で入力方式を切り替える条件を分離しました。これにより、SDKに依存する処理を一か所にまとめ、既存の編集機能へ影響を広げずにマウス操作を追加できる構成としました。

5. 代表的な技術課題と解決方法

5.1 既存の入力・画面表示設計へのマウス操作の統合

課題

既存の入力・画面表示処理は、接続中のプラットフォームやコントローラーの種類を基準に操作方法を決定する設計でした。しかし、Nintendo Switch 2では、同じJoy-Con 2を通常のゲームパッドとして使用する場合と、机上でマウスとして使用する場合があります。そのため、従来の機器判定だけでは現在の操作方法を正しく判断できませんでした。

また、入力判定や操作案内の更新処理が、物体の編集、カメラ、メニューなど複数の機能に分かれて実装されていました。単にマウス入力を追加するだけでは、操作できない機能や表示が切り替わらない画面が残る可能性があったため、既存コード全体から入力方式に依存する箇所を調査し、共通の状態へ追従させる必要がありました。

判断

個々の画面でJoy-Con 2の状態を判定するのではなく、ゲームパッド操作とマウス操作を区別する共通の入力状態を追加しました。入力方式が変化した際に各機能へ通知し、操作処理と画面表示を同じ状態に基づいて更新する方針としました。

実施内容

- 物体の編集、カメラ、メニューなど、入力方式に依存する既存処理を調査し、マウス操作へ対応させました
- 入力方式の変更通知に合わせて、関連する操作案内と画面表示を一括で更新する仕組みを実装しました

結果

分散していた入力判定と画面表示を共通の入力状態へ追従させ、Joy-Con 2の使い方が変わった際に、関連する操作と画面表示を一貫して切り替えられるようにしました。これにより、他プラットフォームを含む既存機能を維持しながら、Nintendo Switch 2版へマウス操作を統合しました。

6. 成果

- 通常のゲームパッド操作とマウス操作を、ユーザー設定、デバイス状態、ゲーム状態に応じて切り替える仕組みを実装しました
- 物体の配置・編集、地形編集、カメラ操作、各種メニュー、画面上の操作案内をマウス操作へ対応させました
- 入力方式を切り替えた後も必要な操作を継続し、二重入力や操作不能を防げる構成を実装しました
- Nintendo Switch 2固有処理を入力基盤へ集約し、ゲーム固有処理との責務を分離しました
- Joy-Con 2の認識から画面上の操作まで、入力の流れを段階的に確認できるデバッグ手段を整備しました

7. 本プロジェクトで得た知識・能力

7.1 Nintendo Switch 2のマウス操作に関する知識と開発経験

Nintendo Switch 2の新機能であるマウス操作について、SDKが提供するデバイス、座標、ボタン割り当て、使用姿勢の判定、ゲームパッド操作との切り替え方法を調査し、マルチプラットフォームタイトルのNintendo Switch 2版へ導入する経験を得ました。

また、SDKから入力を取得するだけでなく、Unityの入力基盤、既存のゲーム操作、メニュー、カメラ、画面上の操作案内まで一連の機能へ統合しました。これにより、他プラットフォームの既存機能を維持しながらNintendo Switch 2固有の入力方式を追加するための設計、実装、影響範囲の調査に関する知識を得ました。

7.2 技術知見の体系化とCEDECでの共有

商用プロジェクトで得たNintendo Switch 2のマウス操作に関する知見を、導入手順、SDK利用時の注意点、入力方式の切り替え、画面表示への対応、共通モジュール化という観点で体系的に整理しました。

さらに、基本機能を確認できるサンプルプロジェクトと講演資料を作成し、クライアントからゲーム映像の使用許諾を得た上で、[CEDEC 2026「Nintendo Switch 2への移植：マウス対応とUnity / Unreal Engine 5におけるパフ](#)

パフォーマンスの最適化」(<https://cedec.cesa.or.jp/2026/timetable/detail/s69f2f05d355d5/>)に共同講演者として登壇しました。講演では、Unityへマウス操作を導入する方法と実プロジェクトで得た設計上の知見を共有しました。この経験を通じて、実装で得た知識を他の開発者が理解・応用できる形へ整理し、技術情報として発信する経験を得ました。

8. 技術的課題と今後の改善

8.1 プレイヤーの意図を考慮した入力方式の切り替え

Joy-Con 2の使用姿勢だけで即座に入力方式を切り替えると、持ち直しなどの一時的な動作を誤って切り替えと判断し、プレイヤーの操作を妨げる可能性があります。今後は、姿勢の継続時間、直前の入力、操作中の状態を組み合わせ、プレイヤーの意図に沿って切り替える仕組みを導入したいと考えています。